

Definición y Aplicación de Políticas de Acceso

Las políticas de acceso son la columna vertebral de la seguridad en entornos cloud. En esta sesión, exploraremos cómo implementar y gestionar políticas de acceso en la nube utilizando técnicas como **RBAC** (Role-Based Access Control) y **ABAC** (Attribute-Based Access Control), herramientas como **AWS IAM Policy Simulator**, y principios como el **privilegio mínimo**. Este documento combina fundamentos teóricos, ejemplos prácticos y referencias técnicas para facilitar la comprensión y aplicación de estos conceptos.

Tipos de Políticas de Acceso: RBAC vs. ABAC

Control de acceso basado en roles (RBAC)

- **Definición** : Modelo que asigna permisos a roles, y roles a usuarios.
- **Ejemplo** :

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Action": "s3:ListBucket",  
7       "Resource": "arn:aws:s3::ejemplo-bucket"  
8     }  
9   ]  
10 }
```

Figura 1: Política RBAC para listar un bucket S3 (AWS, 2023).

1.2 Control de acceso basado en atributos (ABAC)

- **Definición** : Permisos dinámicos basados en atributos (eg, departamento, ubicación).
- **Ejemplo** :

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Action": "s3:GetObject",  
7       "Resource": "arn:aws:s3:::datos-finanzas/*",  
8       "Condition": {  
9         "StringEquals": {  
10          "aws:PrincipalTag/Departamento": "Finanzas"  
11        }  
12      }  
13    }  
14  ]  
15 }
```

Figura 2: Política ABAC para acceso condicional (NIST, 2020).

Comparación RBAC vs. ABAC

CRITERIO	RBAC	ABAC
Granularidad	Roles predefinidos	Atributos dinámicos
Escalabilidad	Limitada	Alta
Complejidad	Baja	Alta

Uso del simulador de políticas de AWS IAM

El **AWS IAM Policy Simulator** es una herramienta esencial para diseñar, probar y depurar políticas de acceso (IAM Policies) de manera segura. Permite simular cómo una política afectaría a un usuario, grupo o rol **antes de aplicarla**, evitando errores que podrían comprometer la seguridad o interrumpir operaciones críticas.

¿Para qué sirve?

- **Validar permisos** : Verificar si una política permite o deniega acciones específicas (ej: ,). **s3:GetObjectec2:StartInstances**
- **Depurar políticas complejas** : Identificar conflictos entre políticas o condiciones mal configuradas.
- **Capacitación** : Entender cómo funcionan las políticas sin riesgo de afectar sistemas reales.

Funcionalidades clave

1. **Simulación de políticas** :
 - Evalúa cómo una política afecta a un usuario/rol en escenarios específicos.
 - Ejemplo: ¿Puede el usuario acceder al bucket S3 ?**juan.perezdatos-finanzas**
2. **Pruebas condicionales** :
 - Evalúa políticas con condiciones como , o etiquetas. **aws:SourceIPaws:UserAgent**
 - Ejemplo: ¿Funciona la política si el usuario se conecta desde una IP no autorizada?
3. **Comparación de políticas** :
 - Analiza diferencias entre versiones de políticas para ajustar permisos.

Pasos para usar el Simulador de Políticas

Paso 1: Acceder al simulador

- Vaya a la **Consola de AWS IAM** → Seleccione **Policy Simulator** en el menú lateral.

Paso 2: Seleccionar entidad y política

- Elegir un usuario, grupo o rol (ej:). **rol-desarrolladores**
- Cargar una política existente o pegar una política JSON personalizada.

Paso 3: Definir la acción a simular

- Seleccione el servicio (ej: S3) y la acción (ej:). **s3:PutObject**
- Especificar el recurso (ej:). **arn:aws:s3::ejemplo-bucket**

Paso 4: Ejecutar la simulación

- El simulador mostrará si la acción es **Permitir** (permitida) o **Denegar** (denegada).
- Explicará qué políticas o condiciones influyeron en el resultado.

Ejemplo práctico : Supongamos que queremos verificar si el rol puede listar objetos en el cubo :

analista-datos ventas-2024

1. Seleccionamos el rol . **analista-datos**
2. Pegamos a la siguiente política:

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Action": "s3:ListBucket",  
7       "Resource": "arn:aws:s3:::ventas-2024"  
8     }  
9   ]  
10 }
```

3. Simulamos la acción sobre el recurso . **s3:ListBucketarn:aws:s3:::ventas-2024**
4. El simulador responde: **Permitir** (¡funciona!).

Ventajas y limitaciones

VENTAJAS	LIMITACIONES
No afecta entornos productivos.	Solo simula políticas, sin recursos.
Soporta políticas en línea y gestionadas.	No hay errores de réplica de red o latencia.
Integra condiciones y contextos (ej: MFA).	Requiere permisos IAM para usarlo.

Mejores prácticas

1. **Usar antes de implementar** : Siempre simula políticas nuevas o modificadas.
2. **Probar escenarios extremos** : Ej: ¿Qué pasa si un usuario no tiene MFA activado?
3. **Resultados documentales** : Guarda capturas de pantalla o registros de las simulaciones.

Ejemplo visual

The screenshot displays the IAM Policy Simulator interface. On the left, the 'Policies' panel shows the 'developer-group-policy' being edited. The policy statement is as follows:

```
{
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "dynamodb:*",
      "Resource": "*"
    }
  ]
}
```

Below the policy editor are 'Apply' and 'Reset' buttons. The main 'Policy Simulator' panel on the right features a grid of AWS services, including Auto Scaling, Marketplace, Website, CloudFormation, CloudFront, CloudWatch, Settings, Direct Connect, DynamoDB, EC2, ElastiCache, Elastic Beanstalk, Elastic Load Balancing, Elastic MapReduce, Glacier, Identity and Access Management, RDS, Redshift, Route 53, S3, SimpleDB, SES, SNS, SQS, and Security Token Service. A 'Run Simulation' button is located in the top right corner of the services grid. At the bottom of the interface, the 'Amazon Web Services' logo is visible.



El **AWS IAM Policy Simulator** es una herramienta indispensable para garantizar que las políticas de acceso funcionen como se espera. Su uso reduce riesgos y ahorra tiempo en la resolución de problemas.

Creación y Administración de Políticas JSON

Las políticas JSON son documentos que definen **qué acciones pueden realizar usuarios, grupos o roles** en servicios de AWS. Su correcta implementación es crítica para garantizar el principio de **privilegio mínimo** y evitar accesos no autorizados.

Estructura de una política JSON

Una política JSON consta de los siguientes elementos clave:

ELEMENTO	DESCRIPCIÓN
Versión	Versión del lenguaje de políticas (ej:). "2012-10-17"
Declaración	Lista de declaraciones que definen reglas de acceso (permisos o prohibiciones).
Efecto	Defina si la acción está permitida () o denegada (). AllowDeny
Acción	Lista de acciones específicas de AWS (ej: ,). s3:GetObjectec2:StartInstances
Recurso	Recursos a los que se aplica la política (ARNs de AWS).
Condición	Opcional: Restricciones basadas en atributos (ej: IP, fecha, etiquetas).

Ejemplo de Política JSON :

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Action": ["s3:GetObject", "s3:ListBucket"],  
7       "Resource": [  
8         "arn:aws:s3:::ejemplo-bucket",  
9         "arn:aws:s3:::ejemplo-bucket/*"  
10      ],  
11      "Condition": {  
12        "IpAddress": {  
13          "aws:SourceIp": "203.0.113.0/24"  
14        }  
15      }  
16    }  
17  ]  
18 }
```

Figura 4: Política que permite acceder a un bucket S3 desde una IP específica.

Pasos para crear una política JSON

1. **Definir el objetivo** : ¿Qué acción(es) debe permitir o bloquear la política?
 - Ejemplo: Acceso de lectura a un bucket S3.
2. **Identificar recursos y acciones** :
 - **Recursos** : ARNs de AWS (ej:). **arn:aws:s3:::mi-bucket**
 - **Acciones** : Lista de operaciones (ej:). **s3:GetObject**
3. **Establecer condiciones (opcional)** :
 - Restricciones como IP, MFA, etiquetas o fechas.
4. **Usar el lenguaje correcto** :
 - Sintaxis JSON estricta (evitar errores de formato).

Mejores Prácticas

1. Privilegio mínimo :

- Solo acciones permitir **estrictamente necesarias** .
- Ejemplo: Si un usuario solo necesita leer objetos en S3, no usar **.s3:***

2. Evitar comodines () innecesarios* :

- **Malo** : → Permite acceso a **todos los recursos** . **"Resource": "*"**
- **Bueno** : → Limita a un cubo específico. **"Resource": "arn:aws:s3::mi-bucket/*"**

3. Usar políticas administradas (AWS Managed Policies) :

- AWS ofrece políticas predefinidas para casos comunes (ej: **AmazonS3ReadOnlyAccess**)

4. Validar con el Simulador de políticas :

- Antes de aplicar, simule la política para evitar errores.

Errores Comunes y Soluciones

ERROR	SOLUCIÓN
Permisos excesivos	Revisar y para reducir el alcance. ActionResource
Sintaxis JSON incorrecta	Usar herramientas como JSONLint para validar.
Conflictos entre políticas	Usar el Simulador de políticas para identificar reglas contradictorias.

Ejemplo Práctico: Política para un Desarrollador

Requisito : Un desarrollador necesita implementar aplicaciones en EC2, pero no debe poder eliminar instancias.

Política JSON :

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Action": [  
7         "ec2:RunInstances",  
8         "ec2:StartInstances",  
9         "ec2:StopInstances"  
10      ],  
11      "Resource": "*"   
12    },  
13    {  
14      "Effect": "Deny",  
15      "Action": "ec2:TerminateInstances",  
16      "Resource": "*"   
17    }  
18  ]  
19 }
```

Figura 5: Política que permite operaciones básicas en EC2 pero bloquea la eliminación de instancias.

Herramientas de apoyo

- **AWS Policy Generator** : Genera políticas JSON mediante una interfaz gráfica. [Enlace al Generador de Políticas](#)
- **AWS CLI** : Para crear políticas desde la línea de comandos:

```
1 aws iam create-policy --policy-name MiPolitica --policy-document file://politica.json
```

Evaluación de Políticas y Principio de Privilegio Mínimo

El **principio de privilegio mínimo** establece que los usuarios, sistemas o servicios deben tener **solo los permisos estrictamente necesarios** para realizar sus funciones, y nada más. Su correcta aplicación reduce el riesgo de accesos no autorizados, errores humanos o ataques internos.

4.1 ¿Qué es el Privilegio Mínimo?

- **Definición** : *"Todo acceso debe ser denegado por defecto, y solo concedido de forma explícita y justificada"* (NIST SP 800-53).
- **Ejemplo** :
 - Un desarrollador que solo necesita implementar aplicaciones en EC2 **no debe tener permisos para eliminar instancias o acceder a buckets S3 de producción** .

4.2 Evaluación de Políticas: ¿Cómo se hace?

Para aplicar el privilegio mínimo, es necesario **auditar y refinar las políticas existentes** . Aquí los pasos clave:

1. **Revisar los permisos actuales** :
 - Identificar acciones () y recursos () permitidos en cada política. **ActionResource**
 - Ejemplo: ¿La política permite (todos los permisos de S3) o solo acciones específicas?**s3:***
2. **Eliminar permisos redundantes** :
 - Usar herramientas como **AWS IAM Access Analyzer** para detectar permisos no utilizados.
 - Ejemplo: Si un usuario no ha usado en 90 días, elimínalo.**ec2:TerminateInstances**

3. Aplicar condiciones estrictas :

- Restringir acciones mediante (ej: IP, MFA, etiquetas). **Condition**
- Ejemplo:

```
1 v "Condition": {  
2 v   "Bool": {  
3     "aws:MultiFactorAuthPresent": "true"  
4   }  
5 }
```

4. Usar el principio de "Denegar por defecto" :

- Si hay conflicto entre políticas, las reglas siempre tienen prioridad sobre **.Deny Allow**

Herramientas para evaluar políticas

1. Analizador de acceso de AWS IAM :

- Detecta permisos excesivos o no utilizados en políticas.
- Genera recomendaciones para aplicar el privilegio mínimo.

2. Simulador de políticas de AWS IAM :

- Permite simular cómo una política afecta a un usuario/rol (como se explica en el punto 2).

3. Configuración de AWS :

- Monitorea el cumplimiento de buenas prácticas (ej: evitar). **Resource": "*"**

Ejemplo Práctico: Corrección de una Política

Antes (política segura) :

```
1 v {  
2   "Version": "2012-10-17",  
3 v   "Statement": [  
4 v     {  
5       "Effect": "Allow",  
6       "Action": "s3:*",  
7       "Resource": "*"   
8     }  
9   ]  
10 }
```

Problemas :

- Permite **todos los permisos de S3** (incluido). **DeleteBucket**
- Aplicar a **todos los recursos** (). **Resource**: "*"

Después (política corregida) :

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": ["s3:GetObject", "s3:ListBucket"],
7       "Resource": [
8         "arn:aws:s3:::ventas-2024",
9         "arn:aws:s3:::ventas-2024/*"
10      ],
11       "Condition": {
12         "IpAddress": {
13           "aws:SourceIp": "203.0.113.0/24"
14         }
15       }
16     }
17   ]
18 }
```

Mejoras :

- Solo permite **leer** en un cubo específico.
- Restrinja el acceso a una IP autorizada.

Errores Comunes y Soluciones

ERROR	SOLUCIÓN
Uso de "Action": "*"	Limitar a acciones específicas (ej:). s3:GetObject
"Resource": "*"	Especificar ARNs de recursos (ej:). arn:aws:s3:::mi-bucket
Falta de condiciones	Agregar para restringir por IP, MFA o etiquetas. Condition

4. Evaluación de Políticas y Principio de Privilegio Mínimo

El **principio de privilegio mínimo** establece que los usuarios, sistemas o servicios deben tener **solo los permisos estrictamente necesarios** para realizar sus funciones, y nada más. Su correcta aplicación reduce el riesgo de accesos no autorizados, errores humanos o ataques internos.

4.1 ¿Qué es el Privilegio Mínimo?

- **Definición** : *"Todo acceso debe ser denegado por defecto, y solo concedido de forma explícita y justificada"* (NIST SP 800-53).
- **Ejemplo** :
 - Un desarrollador que solo necesita implementar aplicaciones en EC2 **no debe tener permisos para eliminar instancias o acceder a buckets S3 de producción** .

4.2 Evaluación de Políticas: ¿Cómo se hace?

Para aplicar el privilegio mínimo, es necesario **auditar y refinar las políticas existentes** . Aquí los pasos clave:

1. **Revisar los permisos actuales** :
 - Identificar acciones () y recursos () permitidos en cada política. **ActionResource**
 - Ejemplo: ¿La política permite (todos los permisos de S3) o solo acciones específicas?**s3:***
2. **Eliminar permisos redundantes** :
 - Usar herramientas como **AWS IAM Access Analyzer** para detectar permisos no utilizados.
 - Ejemplo: Si un usuario no ha usado en 90 días, elimínelo.**ec2:TerminateInstances**

3. Aplicar condiciones estrictas :

- Restringir acciones mediante (ej: IP, MFA, etiquetas). **Condition**
- Ejemplo:

json

```
"Condition": {  
  "Bool": {  
    "aws:MultiFactorAuthPresent": "true"  
  }  
}
```

4. Usar el principio de "Denegar por defecto" :

- Si hay conflicto entre políticas, las reglas siempre tienen prioridad sobre **.DenyAllow**

4.3 Herramientas para evaluar políticas

1. Analizador de acceso de AWS IAM :

- Detecta permisos excesivos o no utilizados en políticas.
- Genera recomendaciones para aplicar el privilegio mínimo.

2. Simulador de políticas de AWS IAM :

- Permite simular cómo una política afecta a un usuario/rol (como se explica en el punto 2).

3. Configuración de AWS :

- Monitorea el cumplimiento de buenas prácticas (ej: evitar). **Resource**: "*"
-

4.4 Ejemplo Práctico: Corrección de una Política

Antes (política segura) :

json

```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Effect": "Allow",  
      "Action": "s3:*",  
      "Resource": "*"   
    }  
  ]  
}
```

Problemas :

- Permite **todos los permisos de S3** (incluido). **DeleteBucket**
- Aplicar a **todos los recursos** (). **Resource**: "*"

Después (política corregida) :

```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Effect": "Allow",  
      "Action": ["s3:GetObject", "s3:ListBucket"],  
      "Resource": [  
        "arn:aws:s3:::ventas-2024",  
        "arn:aws:s3:::ventas-2024/*"  
      ],  
      "Condition": {  
        "IpAddress": {  
          "aws:SourceIp": "203.0.113.0/24"  
        }  
      }  
    }  
  ]  
}
```

Mejoras :

- Solo permite **leer** en un cubo específico.
- Restrinja el acceso a una IP autorizada.

4.5 Errores Comunes y Soluciones

Error	Solución
Uso de "Action": "*" 	Limitar a acciones específicas (ej:).s3:GetObject
"Resource": "*" 	Especificar ARNs de recursos (ej:).arn:aws:s3:::mi-bucket
Falta de condiciones	Agregar para restringir por IP, MFA o etiquetas. Condition

Lista de verificación de Auditoría

1. ¿Los permisos están alineados con el rol del usuario?
2. ¿Se usan comodines () en o ? ***ActionResource**
3. ¿Hay políticas que invalidan reglas ? **DenyAllow**
4. ¿Se han eliminado permisos no utilizados en los últimos 90 días?

La evaluación de políticas y la aplicación del **privilegio mínimo** son esenciales para minimizar riesgos en entornos cloud. Herramientas como **IAM Access Analyzer** y prácticas como la revisión periódica de permisos garantizan un IAM seguro y eficiente.

Auditorías y Revisiones Periódicas

Las auditorías y revisiones periódicas aseguran que las políticas de acceso sigan siendo **relevantes, seguras y alineadas con las necesidades del negocio** . Estas prácticas permiten identificar permisos obsoletos, detectar configuraciones de riesgo y cumplir con normativas como **ISO 27001** , **GDPR** o **SOC 2** .

Objetivos de una Auditoría IAM

1. **Identificar permisos excesivos** : Detectar usuarios o roles con acceso a recursos críticos sin justificación.
2. **Garantizar cumplimiento** : Validar que las políticas cumplen con los estándares internos y externos.
3. **Reducir la superficie de ataque** : Eliminar permisos inactivos o redundantes.
4. **Mejorar la eficiencia operativa** : Simplificar políticas complejas o desactualizadas.

Herramientas para Auditorías en AWS

1. **AWS CloudTrail** :
 - Registra eventos de API (ej: inicio de sesión, cambios en políticas).
 - **Ejemplo de uso** : Identificar quién modificó una crítica política y cuándo.
2. **Configuración de AWS** :
 - Evalúe la configuración de recursos y políticas.
 - **Regla útil** : para detectar políticas con permisos de administrador. **iam-policy-no-statements-with-admin-access**
3. **Analizador de acceso de AWS IAM** :
 - Analiza políticas y genera alertas sobre permisos excesivos.

- **Ejemplo** : Detecta si un rol tiene acceso a buckets S3 de producción sin necesidad.

4. Informe de credenciales de AWS IAM :

- Muestra el estado de las credenciales de usuarios (ej: contraseñas, claves API).
- **Alerta clave** : Usuarios con claves API inactivas desde hace más de 90 días.

Pasos para Realizar una Auditoría

Paso 1: Definir el alcance

- ¿Qué servicios, roles o usuarios se auditarán?
- Ejemplo: Revisar todos los roles asociados al bucket S3 **.datos-finanzas**

Paso 2: Recopilar datos

- Usar **CloudTrail** para obtener registros de actividad.
- Generar un **Informe de Credenciales** para identificar credenciales obsoletas.

Paso 3: Analizar permisos

- Buscar políticas con o . **Resource**": "*" **Action**": "*"
- Usar **IAM Access Analyzer** para detectar permisos no utilizados.

Paso 4: Corregir hallazgos

- Eliminar permisos redundantes.
- Aplicar condiciones (ej: IP, MFA) a políticas críticas.

Paso 5: Documentar y reportear

- Registrar cambios realizados y justificarlos.
- Compartir resultados con equipos de seguridad y cumplimiento.

Práctica: Auditoría de un Rol de Desarrollador

Situación : El rol tiene acceso a EC2 y S3, pero su último uso fue hace 6 meses.
desarrollador-jr

Acciones de auditoría :

1. **CloudTrail** : Verifique si el rol ha realizado acciones recientes.
2. **IAM Access Analyzer** : Confirmar que no tiene permisos no utilizados.
3. **Corrección** :
 - Eliminar acceso a S3 si no es necesario.
 - Restringir EC2 a acciones de solo lectura (). **ec2:DescribeInstances**

Lista de verificación de Auditoría IAM

Tarea	Herramienta recomendada
Identificar permisos inactivos	Analizador de acceso de AWS IAM
Detectar políticas con `Allow: "*"``	Configuración de AWS
Revisar acceso a recursos críticos	AWS CloudTrail
Verificar el uso de MFA en usuarios	Informe de credenciales de AWS IAM
Eliminar claves API antiguas	AWS IAM

Mejores Prácticas

1. **Automatizar revisiones** : Usar **reglas de configuración de AWS** para alertas en tiempo real.
2. **Priorizar riesgos** : Enfocarse primero en roles con acceso a datos sensibles.
3. **Capacitar equipos** : Enseñar a desarrolladores a solicitar solo los permisos necesarios.
4. **Cumplimiento continuo** : Integrar auditorías en ciclos de desarrollo (DevSecOps).

Errores Comunes y Soluciones

ERROR	SOLUCIÓN
Permisos obsoletos	Programar revisión trimestrales para eliminarlos.
Falta de monitoreo	Configurar alarmas en CloudWatch para detección proactiva.
Ignorar cumplimiento	el Usar herramientas como Administrador de auditoría de AWS para informes automatizados.

Las auditorías y revisiones periódicas son un pilar para mantener un IAM seguro y eficiente. Herramientas como **CloudTrail** , **IAM Access Analyzer** y procesos estructurados permiten reducir riesgos y garantizar el cumplimiento normativo. Recuerda: **la seguridad en la nube es un proceso continuo, no un proyecto único** .

Conclusión

Las políticas de acceso son esenciales para mitigar riesgos en la nube. Dominar RBAC, ABAC, herramientas como Policy Simulator y el principio de privilegio mínimo permite implementar un IAM robusto y escalable.

Referencias

- AWS. (2023). *Simulador de políticas de IAM* . Recuperado de <https://aws.amazon.com/iam>
- NIST. (2020). *Control de acceso basado en atributos (ABAC)* . SP 800-162.
- ISO/IEC. (2022). *27001:2022 Gestión de la seguridad de la información* .